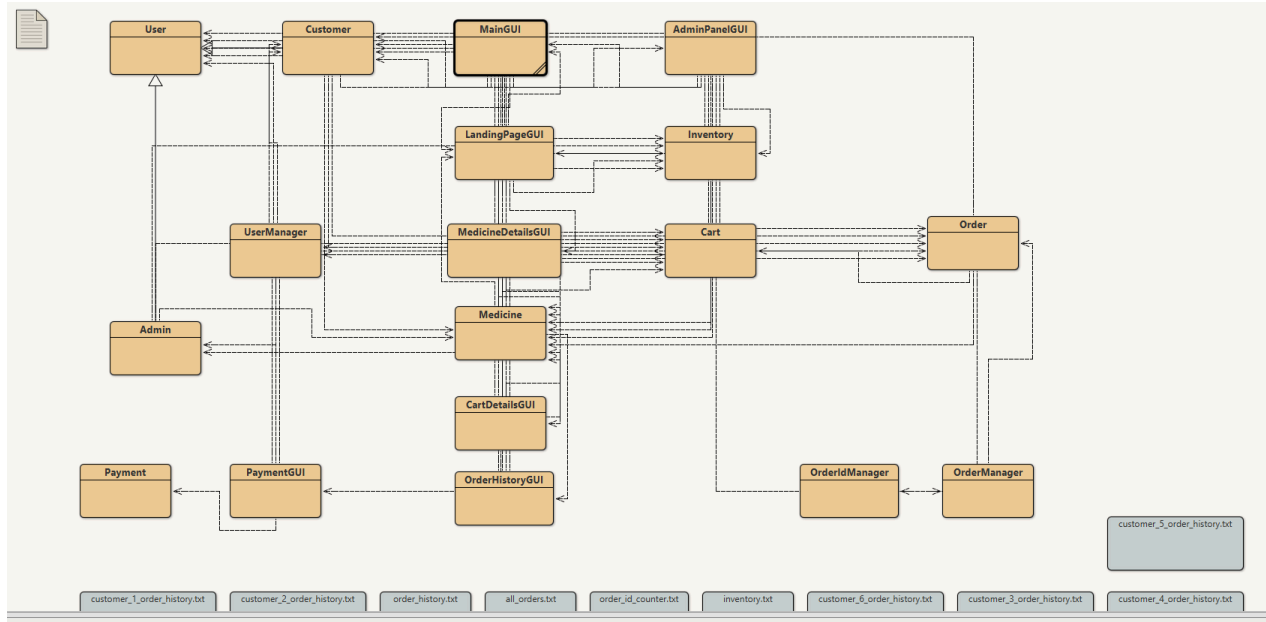


UML Diagram



This is a **UML Class Diagram** for a Java application, likely for an e-commerce or inventory management system. It visually represents the structure of the system, its components (classes), and their relationships. Let's break it down and explain how it connects to the **4 principles of Object-Oriented Programming (OOP)**:

1. Diagram Breakdown

- **Classes and GUIs:**
 - **User:** A base class, likely with attributes like username, password, or roles.
 - **Customer:** Extends the **User** class, showing inheritance (a key OOP principle).
 - **Admin:** May extend **User** or have direct interactions with managing parts of the system.
 - **MainGUI, AdminPanelGUI, LandingPageGUI, CartDetailsGUI, PaymentGUI, OrderHistoryGUI:** These are interfaces for interacting with various aspects of the system.
- **Core Entities:**
 - **Cart, Medicine, Order, Payment, Inventory:** Represent logical entities for managing e-commerce actions.
 - **OrderManager, UserManager, OrderIdManager:** Likely handle backend logic for managing users, orders, or IDs.

- **Relationships:**
 - Lines with arrows represent **associations**, indicating how classes communicate.
 - Inheritance is shown by a hollow triangle (e.g., **Customer** inherits from **User**).
 - **Text Files:**
 - The file names like **order_history.txt** or **inventory.txt** suggest persistence, possibly for storing user or order data.
-

2. Relation to OOP Principles

1. Encapsulation

- **Definition:** Bundling data (fields) and methods (functions) together while restricting direct access.
- **In the Diagram:**
 - Each class is self-contained with responsibilities, like **Cart** for handling items and **OrderManager** for order logic.
 - Likely, access modifiers (**private**, **public**, etc.) are used to protect sensitive data (e.g., cart contents or user credentials).
 - **UserManager** might provide a controlled way to interact with **User** data, enforcing encapsulation.

2. Inheritance

- **Definition:** Allows a class to inherit properties and methods from another.
- **In the Diagram:**
 - **Customer** inherits from **User**, meaning all **User** properties (e.g., name, login) are available to **Customer**.
 - This avoids redundant code and allows for reusability, centralizing shared logic in **User**.

3. Polymorphism

- **Definition:** The ability to process objects differently based on their type.
- **In the Diagram:**
 - Methods in the parent class **User** may be overridden in **Customer** or **Admin** for specific behavior. For example:

- **Admin** might have a specialized method for managing inventory, overriding or extending **User** behavior.
- GUI components like **MainGUI** and **AdminPanelGUI** might implement a common interface for standardization.

4. Abstraction

- **Definition:** Hiding complexity by exposing only essential features.
 - **In the Diagram:**
 - Classes like **UserManager**, **OrderManager**, and **PaymentGUI** likely abstract complex operations like database interactions or payment processing.
 - The use of GUI classes (e.g., **AdminPanelGUI**) abstracts the user interface details, hiding backend logic from the user.
-

3. High-Level Use Cases

- **User Workflow:**
 - A **Customer** logs in through the **MainGUI** and browses products via **LandingPageGUI**.
 - They add items to the **Cart** and proceed to payment via **PaymentGUI**.
 - Their order is created and managed using **OrderManager** and stored in **order_history.txt**.
 - **Admin Workflow:**
 - An **Admin** uses the **AdminPanelGUI** to manage the **Inventory** or view orders via **OrderHistoryGUI**.
-

Main GUI class

This Java class represents the graphical user interface (GUI) for a wellness-related application called **Lumina Wellness**. It uses the **Swing** and **AWT** libraries for creating a window-based interface, handling user inputs, and switching between login and signup forms. Here's an in-depth explanation of the code:

1. Imports and Dependencies

- `javax.swing.*`: For GUI components like buttons, labels, and text fields.
- `java.awt.*`: For layout management and event handling.
- `java.awt.event.*`: To handle user actions like button clicks.
- `java.util.regex.Pattern`: To validate email format using regular expressions (Regex).

2. Class Definition: MainGUI

- **MainGUI** class is the entry point for the GUI of the application. It handles both the login and signup flows, user authentication, and interaction with the backend system (e.g., inventory, cart).

3. Instance Variables

- **frame**: The main window (JFrame) of the application.
- **loginPanel & signupPanel**: Panels for login and signup forms.
- **loginEmailField, signupEmailField, signupNameField**: JTextFields for capturing email and name inputs.
- **loginPasswordField, signupPasswordField**: JPasswordFields for password inputs.
- **loginButton, signupButton, switchToSignupButton, switchToLoginButton**: Buttons for triggering actions like login, signup, and switching between panels.
- **currentUser**: The user currently logged in.
- **userManager**: Manages the list of users, performs authentication, and handles user-related operations.
- **user, inventory, cart, landingPageGUI**: Various objects representing the user, inventory system, shopping cart, and landing page.

4. Constructor: MainGUI()

- Initializes the **JFrame** (`frame`) with a title, size, layout, and close operation.
- Sets the layout of the frame to **CardLayout**, which allows switching between different views (login and signup).
- Initializes the **UserManager**, **Inventory**, **Cart**, and **LandingPageGUI**.
- Calls methods `initializeLoginPanel()` and `initializeSignupPanel()` to set up the respective panels.
- Adds the login and signup panels to the frame.
- Makes the frame visible.

5. Panel Initialization

- **initializeLoginPanel()**: Configures the login form with:
 - **Header Panel**: Displays the logo and brand name.
 - **Login Form Panel**: Contains fields for email and password, and buttons for login and switching to sign up.
 - **Event Handling**: Attaches action listeners for login and switch-to-signup buttons.
- **initializeSignupPanel()**: Configures the signup form with:
 - **Header Panel**: Displays the logo, brand name, and a "SIGNUP" label.
 - **Signup Form Panel**: Contains fields for name, email, password, and buttons for signup and switching to login.
 - **Event Handling**: Attaches action listeners for signup and switch-to-login buttons.

6. Login Handling (`handleLogin()`)

- Retrieves user input (email and password) and validates them.
- If validation fails (empty fields), it displays a message.
- If validation passes, it attempts to authenticate the user through `userManager.validateUser()`.
- If authentication is successful, the user is set as the current user (`currentUser`).
- If the user is a **Customer**, the application switches to the landing page.
- If the user is an **Admin**, it switches to the admin panel.

7. Signup Handling (`handleSignup()`)

- Retrieves user input (name, email, password) and validates:
 - Ensures that all fields are filled.
 - Validates the email format using a regex.
 - Ensures that the password is at least 6 characters long.
- If validation passes, it calls `userManager.addUser()` to add the user to the system.
- If signup is successful, it switches to the login panel; otherwise, it shows an error message.

8. Email Validation (`validateEmail()`)

- A helper method that uses a regular expression to check if the entered email has a valid format (basic pattern for email addresses).

9. Switching Panels (`switchToPanel()`)

- Uses the **CardLayout** to switch between the login and signup panels.
- **CardLayout** is ideal for this case since it allows switching between different "cards" or panels inside the same container.

10. Switching to Landing Page (`switchToLandingPage()`)

- When a customer logs in successfully, the application switches to the **LandingPageGUI**, displaying the main interface for customers after authentication.

11. Switching to Admin Panel (`switchToAdminPanel()`)

- If the user is an admin, the application switches to the **AdminPanelGUI**, which likely handles the administrative features of the system.

12. Main Method (`main()`)

- The `main` method is used to launch the GUI.
 - `SwingUtilities.invokeLater(MainGUI::new)` ensures that the GUI creation is done on the Event Dispatch Thread (EDT), which is the proper thread for GUI updates in Swing.
-

Summary:

The `MainGUI` class creates a graphical user interface for a wellness-related application. It allows users to log in or sign up, validates their inputs (email, password), and then navigates them to either a landing page (for customers) or an admin panel (for administrators). The application uses **Swing** for the GUI and employs **CardLayout** to switch between the login and signup panels. Various actions such as logging in, signing up, and switching panels are handled by adding event listeners to buttons and validating user inputs.

Landing Page GUI

This Java class, `LandingPageGUI`, is part of a graphical user interface (GUI) for an application related to a wellness store (Lumina Wellness). It displays a landing page for customers that includes a list of medicines, options for viewing the cart and order history, and a logout button. Let's break it down step-by-step:

Overview:

- **Purpose:** The class manages the landing page for customers after logging in. It shows available medicines, provides options to view the cart and order history, and allows the user to log out.
- **Components:**
 - `JPanel`: Main container for the interface.
 - `Inventory`: Holds available medicines.
 - `Cart`: Holds the customer's shopping cart.
 - `User`: Represents the logged-in user.

Key Elements:

1. Instance Variables:

- `panel`: Main container panel for the landing page.
- `medicinesPanel`: Panel to display the medicines.
- `inventory`: Reference to the `Inventory` class, which manages available medicines.
- `cart`: Reference to the `Cart` class, which contains the user's cart.

2. Constructor (**LandingPageGUI**)

- **Inputs:** **Inventory** and **Cart** objects are passed to the constructor.
- **Logic:**
 - If no user is logged in (i.e., `cart.getUser()` is `null`), a warning message is shown.
 - Initializes the main `panel` using `BorderLayout`, and adds sections like header, medicine list, and footer.
 - The `updateMedicinesPanel()` method is called to populate the list of medicines.

3. Header Section (**createHeader**)

- This section is a simple `JPanel` containing a `JLabel` with the title "Lumina Medicines".
- The title is centered and styled with a bold font.

4. Medicine Display (**createMedicineScrollPanel**)

- A scrollable panel (`JScrollPane`) is used to display medicines in a grid layout (`GridLayout`).
- The `medicinesPanel` is populated dynamically with medicine cards that can be scrolled.

5. Footer Section (**createFooter**)

- This section contains three buttons: "View Cart", "Order History", and "Logout".
- Each button is linked to a corresponding action (e.g., opening the cart details, viewing order history, or logging out).

6. Medicines Panel (**updateMedicinesPanel**)

- This method populates `medicinesPanel` with a list of medicines fetched from the inventory.
- It dynamically creates a "card" (a small panel) for each medicine, displaying the name, price, stock, and an image (if available).

7. Medicine Card (**createMedicineCard**)

- For each medicine in the inventory, a card-like interface is created.
- Each card contains:
 - An image (if available, otherwise text saying "No Image").
 - Medicine name, price, and stock.
 - Two buttons: "Add to Cart" and "View Details".
 - **Add to Cart Button:** When clicked, it checks the stock and adds the medicine to the cart if it's available, updating the stock.
 - **View Details Button:** Opens a new window showing detailed information about the medicine (**MedicineDetailsGUI**).

8. Handle Add to Cart (**handleAddToCart**)

- This method adds the selected medicine to the cart and reduces the stock by 1.
- If the medicine is out of stock, an error message is shown.

9. Order History (**handleOrderHistory**)

- This method opens the **OrderHistoryGUI** if the logged-in user is a **Customer**.
- If the user is not a **Customer**, an error message is shown.

10. Logout (**logout**)

- Logs the user out by switching to the main GUI (**MainGUI**), clearing the current frame, and creating a new instance of **MainGUI**.

11. Button Creation (**createButton**)

- This helper method creates a styled button with a given text and action listener. The button has a dark grey background and white text.

12. Getting the Main Panel (**getPanel**)

- The **getPanel()** method returns the main panel of the landing page, which can be added to a frame or another container.

Flow of Execution:

1. **Login:** When the user logs in, they are taken to the landing page, where they can browse available medicines.
2. **Browse Medicines:** Each medicine is shown with details and options to add it to the cart or view more details.
3. **Cart and History:** The user can view their cart, order history (if they are a customer), or log out.
4. **Logout:** When the user logs out, they are taken back to the login screen (`MainGUI`).

Key Methods:

- `updateMedicinesPanel()`: Dynamically updates the display of medicines in the grid.
- `handleAddToCart(Medicine medicine)`: Handles the logic of adding a medicine to the cart and updating stock.
- `handleOrderHistory(User user)`: Opens the order history page for customers.
- `logout()`: Logs the user out and takes them back to the login screen.

Overall Design:

- The class makes extensive use of `JPanel`, `JScrollPane`, `JButton`, and `JLabel` to create a rich, interactive GUI.
- Event handling is managed using `ActionListener` interfaces attached to buttons.
- The GUI layout is flexible with components arranged in `BorderLayout`, `FlowLayout`, and `GridLayout` for responsiveness.
- It integrates functionality such as login validation, cart management, and order history viewing.

This class is a key component of a customer-facing interface in a wellness store application.

Medicine Details GUI

The `MedicineDetailsGUI` Java class is responsible for creating and displaying a detailed view of a specific medicine, allowing the user to see more information and add the medicine to their shopping cart. It uses Swing components to build the graphical user interface (GUI). Let's break down the class and its functionality:

Class Overview:

- **Purpose:** Displays detailed information about a particular medicine, including its image, price, stock, description, and provides an option for the user to add it to their shopping cart.
- **Components:** It creates a window (`JFrame`) that contains panels (`JPanel`) to organize and display different sections like the title, details, description, and buttons.

Key Elements:

1. Constructor (`MedicineDetailsGUI`):

- **Inputs:**
 - `medicine`: The specific medicine whose details need to be displayed.
 - `cart`: The user's shopping cart where the selected medicine can be added.
- **Logic:**
 - It creates a new `JFrame` for the details page.
 - The layout of the frame is set using `BorderLayout`, and the main sections of the page (title, medicine details, and bottom actions) are added to the frame.

2. Creating the Frame (`createFrame`):

- **Purpose:** Creates a new `JFrame` that will hold all the components for the medicine details page.
- **Details:**
 - The frame title is set to the medicine's name followed by " - Details".
 - The size of the frame is fixed (600x500 pixels).
 - The default close operation is set to `DISPOSE_ON_CLOSE`, meaning the frame will be closed when the user closes the window.
 - The frame is centered on the screen using `frame.setLocationRelativeTo(null)`.

3. Creating the Main Panel (**createMainPanel**):

- **Purpose:** Creates a **JPanel** with a **BorderLayout** to organize the layout of the window's content.
- **Details:**
 - A border is added around the main panel with some padding (20 pixels on each side).
 - The main panel is added to the frame, and the frame is displayed.

4. Title Panel (**createTitlePanel**):

- **Purpose:** Displays the name of the medicine at the top of the window.
- **Details:**
 - A **JLabel** is created with the name of the medicine, which is centered and styled using the "Segoe UI" font with a bold weight.
 - The label's color is set to dark gray.

5. Details Panel (**createDetailsPanel**):

- **Purpose:** Contains sections for the medicine's image, price, stock, and description.
- **Subsections:**
 - **Image:** Displays the image of the medicine (if available). If no image is provided, a placeholder text ("No Image") is displayed.
 - **Info:** Shows the medicine's price and stock. This is done using a **JPanel** with a **BoxLayout** to vertically stack the labels.
 - **Description:** Displays a scrollable description of the medicine, using a **JTextArea** inside a **JScrollPane** for easy reading of larger descriptions.

6. Creating the Image Label (**createImageLabel**):

- **Purpose:** Creates a **JLabel** to display the medicine's image, scaled to fit a size of 150x150 pixels.
- **Details:**
 - If the **medicine** has an image path, it is loaded and displayed as an **ImageIcon**. If not, a placeholder text ("No Image") is shown.
 - The image is scaled smoothly to fit the label.

7. Creating the Info Panel (`createInfoPanel`):

- **Purpose:** Displays the price and stock information of the medicine.
- **Details:**
 - The price and stock are displayed using `JLabels`.
 - The `styleInfoLabel` method is used to apply consistent styling to the labels (font size, color, etc.).

8. Creating the Description Scroll Pane (`createDescriptionScrollPane`):

- **Purpose:** Displays a scrollable area with the medicine's description.
- **Details:**
 - A `JTextArea` is used to show the description, which is non-editable, line-wrapped, and has a smooth user experience when scrolling.
 - A border is added to the `JScrollPane` to visually separate it from other sections.

9. Bottom Panel (`createBottomPanel`):

- **Purpose:** Contains user input elements (quantity) and the "Add to Cart" button.
- **Subsections:**
 - **Quantity Input:** A `JTextField` allows the user to input the quantity of the medicine they wish to add to the cart.
 - **Add to Cart Button:** A button that, when clicked, adds the medicine to the shopping cart if the quantity is valid (positive and within available stock).

10. Creating the "Add to Cart" Button (`createAddToCartButton`):

- **Purpose:** Creates a button that allows the user to add the selected medicine to the shopping cart.
- **Logic:**
 - The button has an `ActionListener` that checks the input quantity, ensures it is a valid number, and verifies that the medicine is in stock.
 - If the quantity is valid and in stock, the medicine is added to the cart, the stock is updated, and a success message is shown.
 - If the quantity is invalid or the medicine is out of stock, an error message is shown.

11. Message Handling:

- The class makes extensive use of `JOptionPane` to display informational and error messages (e.g., when a medicine is added to the cart or when there is an invalid input).

Flow of Execution:

1. User Interaction:

- The user selects a medicine to view its details.
- The `MedicineDetailsGUI` window is displayed with the medicine's name, image (if available), price, stock, and description.
- The user can enter a quantity and click "Add to Cart" to add the medicine to their cart.

2. Adding to Cart:

- The application verifies that the input quantity is valid and checks the stock level. If everything is correct, the medicine is added to the cart, the stock is updated, and a success message is shown.
- If there is an issue (invalid quantity, out of stock, etc.), an appropriate error message is shown.

3. Window Closure:

- After adding to the cart, the details window is closed automatically (`frame.dispose()`).

Key Methods:

- `createFrame()`: Creates and configures the main window (`JFrame`).
- `createMainPanel()`: Sets up the layout for the main window.
- `createTitlePanel()`: Displays the medicine's name.
- `createDetailsPanel()`: Contains sections for the image, price/stock info, and description.
- `createImageLabel()`: Displays the medicine's image.
- `createInfoPanel()`: Displays price and stock information.
- `createDescriptionScrollPane()`: Displays a scrollable description of the medicine.
- `createBottomPanel()`: Displays quantity input and the "Add to Cart" button.
- `createAddToCartButton()`: Handles adding the medicine to the cart.

Summary:

The `MedicineDetailsGUI` class is a well-structured window that displays detailed information about a specific medicine. It allows users to view the medicine's image, price, stock, and description, and provides an option to add the medicine to the cart. This is a crucial part of a larger e-commerce or inventory system where users can browse products, see details, and make purchases.

Medicine Class

Explanation of the `Medicine` Class:

The `Medicine` class in Java is designed to represent a medicine product with various properties such as its unique ID, name, price, description, components, side effects, stock, and image path. Additionally, the class includes functionality to manage the stock, convert the object to and from CSV format, and store or transmit the object using Java serialization.

Key Features of the `Medicine` Class:

1. `Serializable`:

- The class implements the `Serializable` interface, which allows instances of the class to be converted into a byte stream. This enables objects of the `Medicine` class to be written to a file or sent over a network. The `serialVersionUID` field is used for version control in serialization.

2. `Fields (Attributes)`:

- The `Medicine` class has the following fields:
 - `medicineId`: A unique identifier for the medicine.
 - `name`: The name of the medicine.
 - `price`: The price of the medicine.
 - `description`: A detailed description of the medicine.
 - `components`: The ingredients or components of the medicine.
 - `sideEffects`: Possible side effects from using the medicine.
 - `stock`: The quantity of the medicine available in stock.
 - `imagePath`: The file path to an image of the medicine.

3. Constructor:

- The constructor initializes all the fields of the `Medicine` object. It takes the following parameters:

Java

```
public Medicine(int medicineId, String name, double price, String description, String components, String sideEffects, int stock, String imagePath) { ... }
```

4. CSV Conversion Methods:

- `toCSV()`:
 - Converts the `Medicine` object to a CSV (Comma-Separated Values) formatted string. This is useful for saving or transmitting medicine data.
 - It ensures that any commas in the attributes (like `name`, `description`, etc.) are escaped using `,` to prevent them from breaking the CSV structure.

Java

```
public String toCSV() { ... }
```

- `fromCSV(String csv)`:
 - A static method that takes a CSV string and returns a `Medicine` object. It splits the CSV string by commas, unescapes the commas that were replaced by `,`, and then creates a new `Medicine` object from the parsed data.

Java

```
public static Medicine fromCSV(String csv) { ... }
```

5. Stock Management:

- `updateStock(int quantity)`:
 - Updates the stock by adding or subtracting a specified quantity. A positive number increases the stock, while a negative number decreases it.

Java

```
public void updateStock(int quantity) { ... }
```

- **removeStock(int quantity):**
 - Reduces the stock by the specified quantity. This method is essentially a shorthand for calling `updateStock` with a negative value.

Java

```
public void removeStock(int quantity) { ... }
```

6. Details and Availability:

- **getDetails():**
 - Returns a detailed string representation of the medicine's attributes (name, price, description, components, side effects, stock, and image path). This is useful for displaying the full details of the medicine.

Java

```
public String getDetails() { ... }
```

- **isAvailable(int quantity):**
 - Checks if the requested quantity of the medicine is available in stock. Returns `true` if the stock is sufficient; otherwise, it returns `false`.

Java

```
public boolean isAvailable(int quantity) { ... }
```

7. **Getters and Setters:** The class provides getter and setter methods for all its fields. These methods allow other classes to retrieve or modify the values of the `Medicine` object's attributes.

- **Getters:**
 - `getMedicineId():` Returns the `medicineId`.
 - `getName():` Returns the `name` of the medicine.
 - `getPrice():` Returns the `price` of the medicine.

- `getStock()`: Returns the `stock` of the medicine.
- `getImagePath()`: Returns the path to the image file of the medicine.
- `getDescription()`: Returns the `description` of the medicine.
- **Setters:**
 - `setPrice(double price)`: Sets the `price` of the medicine.
 - `setStock(int stock)`: Sets the `stock` of the medicine.
 - `setDescription(String description)`: Sets the `description` of the medicine.
 - `setComponents(String components)`: Sets the `components` of the medicine.
 - `setSideEffects(String sideEffects)`: Sets the `sideEffects` of the medicine.
 - `setImagePath(String imagePath)`: Sets the `imagePath` of the medicine.

Detailed Breakdown of Important Methods:

`toCSV()`:

This method converts the `Medicine` object into a CSV-formatted string, making it suitable for saving to a file or transmitting it. It ensures that commas inside string fields are escaped to prevent breaking the CSV format.

Example:

Java

```
public String toCSV() {  
  
    return String.format("%d,%s,%.2f,%s,%s,%s,%d,%s",  
  
        medicineId,  
  
        name.replace(",", ";&#44;"),  
  
        price,  
  
        description.replace(",", ";&#44;"),
```

```
        components.replace(",", " ", "&#44;"),  
        sideEffects.replace(",", " ", "&#44;"),  
        stock,  
        imagePath.replace(",", " ", "&#44;"));  
    }  
}
```

fromCSV(String csv):

This static method takes a CSV-formatted string, parses it, and creates a new **Medicine** object using the extracted values. It also handles unescaping the commas that were replaced by **,** during the CSV export.

Example:

Java

```
public static Medicine fromCSV(String csv) {  
    String[] parts = csv.split(",");  
    if (parts.length == 8) {  
        return new Medicine(  
            Integer.parseInt(parts[0]),  
            parts[1].replace("&#44;", " ,"),  
            Double.parseDouble(parts[2]),  
            parts[3].replace("&#44;", " ,"),  
            parts[4].replace("&#44;", " ,"),  
            parts[5].replace("&#44;", " ,"),  
            parts[6].replace("&#44;", " ,"),  
            parts[7].replace("&#44;", " ,")  
        );  
    }  
}
```

```

        parts[5].replace("&#44;", ", "),
        Integer.parseInt(parts[6]),
        parts[7].replace("&#44;", ", ")
    );
}

        throw new IllegalArgumentException("Invalid CSV format for
Medicine");
}

```

updateStock(int quantity) and removeStock(int quantity):

These methods allow you to update the stock level by either adding or removing quantities. The `updateStock()` method can both increase and decrease stock, while `removeStock()` is simply a wrapper around `updateStock()` to remove stock.

```

Java
public void updateStock(int quantity) {

    this.stock += quantity;

}

public void removeStock(int quantity) {

    this.stock -= quantity;

}

```

getDetails():

Returns a detailed string that includes the key properties of the medicine, formatted for easy display. This method can be used in a UI to show users the full details of the medicine.

Example:

Java

```
public String getDetails() {  
  
    return String.format("Name:  %s\nPrice:  PHP%.2f\nDescription:  
%s\nComponents: %s\nSide Effects: %s\nStock: %d\nImage Path: %s",  
  
        name, price, description, components, sideEffects, stock,  
        imagePath);  
  
}
```

Usage Example:

Java

```
// Create a Medicine object  
  
Medicine medicine = new Medicine(101, "Paracetamol", 5.50, "Pain  
reliever", "Paracetamol", "May cause drowsiness", 100,  
"/images/paracetamol.jpg");  
  
// Convert the Medicine object to CSV format  
  
String csv = medicine.toCSV();  
  
System.out.println(csv); // Output CSV string
```

```
// Create a new Medicine object from the CSV string

Medicine newMedicine = Medicine.fromCSV(csv);

System.out.println(newMedicine.getDetails()); // Output the details of
the new Medicine object
```

Conclusion:

The `Medicine` class is a well-structured representation of a medicine with various attributes like price, stock, description, and more. It also includes features for managing the stock, serializing to/from CSV format, and ensuring that object data can be transferred or saved in a consistent way. This class is useful in scenarios such as e-commerce systems, inventory management, or any application that requires managing medicine data.

Cart Details GUI

Explanation of the `CartDetailsGUI` Class:

The `CartDetailsGUI` class is part of a Java Swing-based GUI (Graphical User Interface) that provides a user interface for displaying and managing the contents of a user's shopping cart. It allows the user to view the cart, remove items, and proceed to checkout.

The GUI is organized into several components, including labels, buttons, and panels, which are structured and displayed in a `JFrame` window. This class is typically used in an application that deals with shopping carts, such as an e-commerce or inventory management system.

Key Features of the `CartDetailsGUI` Class:

1. **Fields (Attributes):**

- **frame**: The main **JFrame** window where the cart details are displayed.
- **cart**: An instance of the **Cart** class, which holds the items in the user's shopping cart.
- **inventory**: An instance of the **Inventory** class, which keeps track of the available inventory and can update stock when items are removed from the cart.

2. **Constructor**: The constructor takes two parameters:

- **cart**: The shopping cart to display.
- **inventory**: The inventory system that can be updated when items are removed from the cart.

3. It initializes the cart and inventory and then sets up the main window (**JFrame**) using the **initializeFrame** method.

Java

```
public CartDetailsGUI(Cart cart, Inventory inventory) {  
  
    this.cart = cart;  
  
    this.inventory = inventory;  
  
    initializeFrame();  
  
}
```

4. **Main Window (JFrame) Setup**: The main window is created and configured in the **initializeFrame()** method. It sets the size of the window, the layout, and the components that will be added to the window.

Java

```
private void initializeFrame() {  
  
    frame = new JFrame("Cart Details");  
  
    frame.setSize(600, 500);  
  
    frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
  
}
```

```

frame.setLayout(new BorderLayout());

frame.add(createTitleLabel(), BorderLayout.NORTH);

frame.add(createCartScrollPane(), BorderLayout.CENTER);

frame.add(createBottomPanel(), BorderLayout.SOUTH);

frame.setVisible(true);

}

```

5. **Title Label:** The `createTitleLabel()` method creates a centered label at the top of the window to display the title "Your Cart". This label is placed at the `BorderLayout.NORTH` position.

Java

```

private JLabel createTitleLabel() {

    JLabel titleLabel = new JLabel("Your Cart", JLabel.CENTER);

    titleLabel.setFont(new Font("Serif", Font.BOLD, 24));

    return titleLabel;

}

```

6. **Scroll Pane for Cart Items:** The cart contents are displayed in a scrollable panel. The `createCartScrollPane()` method creates a `JPanel` with a `BoxLayout` to arrange items vertically and adds it to a `JScrollPane` for scrolling capability.

Java

```

private JScrollPane createCartScrollPane() {

```

```

JPanel cartPanel = new JPanel();

cartPanel.setLayout(new BoxLayout(cartPanel, BoxLayout.Y_AXIS));

updateCartPanel(cartPanel);

return new JScrollPane(cartPanel);

}

```

7. **Bottom Panel with Checkout Button:** The bottom of the window contains a panel with a "Checkout" button. The `createBottomPanel()` method creates a panel with the button, and adds an `ActionListener` to handle the checkout process.

Java

```

private JPanel createBottomPanel() {

    JPanel bottomPanel = new JPanel(new FlowLayout(FlowLayout.CENTER));

    JButton checkoutButton = new JButton("Checkout");

    checkoutButton.addActionListener(e -> handleCheckout());

    bottomPanel.add(checkoutButton);

    return bottomPanel;

}

```

8. **Updating Cart Panel:** The `updateCartPanel()` method updates the display of the cart items. If the cart is empty, it adds a label saying "Your cart is empty." Otherwise, it loops through the list of medicines in the cart and adds each one to the panel using `createItemPanel()`.

Java

```
private void updateCartPanel(JPanel cartPanel) {

    cartPanel.removeAll();

    if (cart.getMedicines().isEmpty()) {

        cartPanel.add(new JLabel("Your cart is empty.", JLabel.CENTER));

    } else {

        for (Medicine medicine : cart.getMedicines()) {

            cartPanel.add(createItemPanel(medicine));

        }

    }

    cartPanel.revalidate();

    cartPanel.repaint();

}
```

9. **Creating Item Panels:** Each item in the cart is displayed in a panel created by the `createItemPanel()` method. This panel includes:

- A label displaying the medicine's image (if available).
- A panel with the medicine's name, price, and quantity.
- A panel with a "Remove" button to remove the item from the cart.

Java

```
private JPanel createItemPanel(Medicine medicine) {

    JPanel itemPanel = new JPanel(new BorderLayout(10, 10));
```

```

        itemPanel.setBorder(BorderFactory.createLineBorder(new Color(200,
200, 200), 1));

        itemPanel.setBackground(new Color(250, 250, 250));

        itemPanel.add(createImageLabel(medicine), BorderLayout.WEST);

        itemPanel.add(createInfoPanel(medicine), BorderLayout.CENTER);

        itemPanel.add(createButtonPanel(medicine), BorderLayout.SOUTH);

        return itemPanel;
    }

```

10. Displaying Medicine Image: The `createImageLabel()` method attempts to load an image from the medicine's image path. If an image is available, it is displayed at a fixed size. If no image is available, a text label "No Image" is shown instead.

Java

```

private JLabel createImageLabel(Medicine medicine) {

    JLabel imageLabel = new JLabel();

    if (medicine.getImagePath() != null &&
!medicine.getImagePath().isEmpty()) {

        ImageIcon icon = new ImageIcon(new
ImageIcon(medicine.getImagePath())

```

```

        .getImage().getScaledInstance(100, 100,
Image.SCALE_SMOOTH));

        imageLabel.setIcon(icon);

    } else {

        imageLabel.setText("No Image");

        imageLabel.setHorizontalAlignment(JLabel.CENTER);

    }

    return imageLabel;

}

```

11. Displaying Medicine Details: The `createInfoPanel()` method creates a panel with the medicine's name, price, and quantity in the cart. The labels are displayed in a vertical layout (using `GridLayout`).

Java

```

private JPanel createInfoPanel(Medicine medicine) {

    JPanel infoPanel = new JPanel(new GridLayout(3, 1, 5, 5));

    infoPanel.setOpaque(false);

    JLabel nameLabel = new JLabel(medicine.getName());

    nameLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));

    JLabel priceLabel = new JLabel("Price: PHP " + medicine.getPrice());

```

```

        JLabel quantityLabel = new JLabel("Quantity: " +
cart.getQuantity(medicine));

        infoPanel.add(nameLabel);

        infoPanel.add(priceLabel);

        infoPanel.add(quantityLabel);

        return infoPanel;
    }

```

12. **Remove Button:** Each item has a "Remove" button that, when clicked, removes the medicine from the cart, updates the stock in the inventory, and refreshes the cart display. This is handled by the `createButtonPanel()` and `handleRemove()` methods.

Java

```

private JPanel createButtonPanel(Medicine medicine) {

    JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 5,
5));

    JButton removeButton = new JButton("Remove");

    removeButton.addActionListener(e -> handleRemove(medicine));

    buttonPanel.add(removeButton);
}

```

```
        return buttonPanel;
    }
}
```

13. Checkout Process: The `handleCheckout()` method is triggered when the user clicks the "Checkout" button. If the cart is empty, it shows an error message. Otherwise, it proceeds to a `PaymentGUI` to handle the payment process.

Java

```
private void handleCheckout() {

    if (cart.getMedicines().isEmpty()) {

        JOptionPane.showMessageDialog(frame, "Your cart is empty!");

        return;

    }

    new PaymentGUI(cart.calculateTotalPrice(), cart);

    frame.dispose();

}
```

14. Handling Item Removal: When an item is removed from the cart, the `handleRemove()` method is triggered. It removes the item from the cart, updates the inventory, and refreshes the cart display.

Java

```
private void handleRemove(Medicine medicine) {
```

```

    int quantityRemoved = cart.getQuantity(medicine);

    cart.removeMedicine(medicine);

    medicine.updateStock(quantityRemoved);

    inventory.saveInventoryToFile();

    JOptionPane.showMessageDialog(null, medicine.getName() + "
successfully removed from your cart.", "Item Removed",
JOptionPane.INFORMATION_MESSAGE);

    updateCartPanel((JPanel) frame.getContentPane().getComponent(1)); //
Refresh the cart panel.

}

```

Summary:

The `CartDetailsGUI` class is a graphical user interface for managing a shopping cart in a Java application. It provides functionality to display the cart items, show their details (including images, names, prices, and quantities), allow the user to remove items, and proceed to checkout. The GUI is organized into several panels, including a main title, a scrollable cart area, and a bottom panel with a checkout button.

Order History GUI

The `OrderHistoryGUI` class is part of a Java Swing application that allows customers to view and manage their order history. It displays the customer's previous orders in a graphical interface, where

the user can see details about each order and perform actions like tracking or canceling orders. Below is an explanation of each component of the code:

Class Overview

- The `OrderHistoryGUI` class is responsible for creating a graphical interface where the user can view and manage their order history.
- The main user interactions in this GUI are:
 1. Viewing past orders.
 2. Tracking the status of an order.
 3. Cancelling an order (if it's not already processed).

Attributes (Fields)

1. **frame:**
 - A `JFrame` object, which is the main window of the application where the order history is displayed.
2. **customer:**
 - A `Customer` object, which represents the customer whose order history is being shown. This allows the application to retrieve the customer's order history and display it.

Constructor

Java

```
public OrderHistoryGUI(Customer customer) {  
  
    this.customer = customer; // Assign the customer to the instance  
    variable.  
  
    initializeFrame(); // Set up the frame and layout.  
  
    populateOrderHistory(); // Populate the order history.  
  
}
```

- This constructor takes a `Customer` object and initializes the `OrderHistoryGUI`. It sets up the main frame (`initializeFrame()`) and populates the order history panel (`populateOrderHistory()`).

Methods

1. `initializeFrame()`

Java

```
private void initializeFrame() {  
  
    frame = new JFrame("Order History");  
  
    frame.setSize(600, 400);  
  
    frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
  
    frame.setLayout(new BorderLayout());  
  
    JLabel titleLabel = new JLabel("Order History", JLabel.CENTER);  
  
    titleLabel.setFont(new Font("Serif", Font.BOLD, 24));  
  
    frame.add(titleLabel, BorderLayout.NORTH);  
  
}
```

- This method sets up the main window (`JFrame`) for the order history.
 - **Title:** A centered label ("Order History") is added at the top of the frame using `BorderLayout.NORTH`.
 - The window is configured to be 600x400 pixels, and the `DISPOSE_ON_CLOSE` option ensures that the window is disposed of when closed.

2. `populateOrderHistory()`

Java

```
private void populateOrderHistory() {

    JPanel historyPanel = new JPanel();

        historyPanel.setLayout(new    BoxLayout(historyPanel,
BoxLayout.Y_AXIS)); // Vertical layout for orders.

    ArrayList<Order> orders = customer.getOrderHistory();

    if (orders.isEmpty()) {

        JLabel noOrdersLabel = new JLabel("No orders found.",
JLabel.CENTER);

        noOrdersLabel.setFont(new Font("Arial", Font.PLAIN, 16));

        historyPanel.add(noOrdersLabel);

    } else {

        for (Order order : orders) {

            JPanel orderPanel = createOrderPanel(order);

            historyPanel.add(orderPanel);

        }

    }

    JScrollPane scrollPane = new JScrollPane(historyPanel);

    frame.add(scrollPane, BorderLayout.CENTER);
```

```
frame.setVisible(true);  
  
}
```

- This method creates and populates the main panel that displays the order history:
 - **Empty Orders:** If the customer has no orders, it displays a message "No orders found."
 - **Orders:** If there are orders, it creates a panel for each order using the `createOrderPanel()` method, and adds it to the `historyPanel`.
 - **Scroll Pane:** The order history panel is wrapped in a `JScrollPane` to make the list scrollable if there are many orders.

3. `createOrderPanel(Order order)`

Java

```
private JPanel createOrderPanel(Order order) {  
  
    JPanel orderPanel = new JPanel(new BorderLayout());  
  
    orderPanel.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));  
  
    JTextArea orderDetails = new JTextArea(order.toString());  
  
    orderDetails.setEditable(false);  
  
    orderPanel.add(new JScrollPane(orderDetails), BorderLayout.CENTER);  
  
    JPanel buttonPanel = new JPanel(new FlowLayout());  
  
    JButton trackButton = createTrackButton(order);  
  
    JButton cancelButton = createCancelButton(order);  
  
}
```

```

buttonPanel.add(trackButton);

if (!"Canceled".equals(order.getStatus())) {

    buttonPanel.add(cancelButton);

}

orderPanel.add(buttonPanel, BorderLayout.SOUTH);

return orderPanel;

}

```

- This method creates a panel displaying the details of a specific order:
 - **Order Details:** A `JTextArea` is used to display the string representation of the order (`order.toString()`).
 - **Buttons:** It adds two buttons (`Track` and `Cancel`) at the bottom of the panel.
 - The `Cancel` button is only shown if the order hasn't been canceled already.
 - **Return:** Returns a `JPanel` containing the order details and buttons.

4. `createTrackButton(Order order)`

Java

```

private JButton createTrackButton(Order order) {

    JButton trackButton = new JButton("Track");

```

```

                                trackButton.addActionListener(e    ->
JOptionPane.showMessageDialog(frame,    "Order    Status:    "    +
order.getStatus()));

    return trackButton;

}

```

- This method creates a "Track" button that shows the status of the order in a message dialog when clicked.

5. createCancelButton(Order order)

Java

```

private JButton createCancelButton(Order order) {

    JButton cancelButton = new JButton("Cancel");

    cancelButton.addActionListener(new CancelOrderListener(order,
customer));

    return cancelButton;

}

```

- This method creates a "Cancel" button that, when clicked, triggers the cancellation of the order. It uses the `CancelOrderListener` inner class to handle the cancel action.

Inner Class: `CancelOrderListener`

The `CancelOrderListener` class handles the logic of canceling an order when the "Cancel" button is clicked.

1. Constructor

Java

```
public CancelOrderListener(Order order, Customer customer) {  
  
    this.order = order;  
  
    this.customer = customer;  
  
}
```

- The constructor initializes the `CancelOrderListener` with the `Order` to be canceled and the `Customer` who placed the order.

2. `actionPerformed(ActionEvent e)`

Java

@Override

```
public void actionPerformed(ActionEvent e) {  
  
    if (!"Processed".equals(order.getStatus())) {  
  
        // Add the medicine stock back to the inventory.  
  
        for (Medicine medicine : order.getCart().getMedicines()) {  
  
            int quantity = order.getCart().getQuantity(medicine);  
  
            medicine.updateStock(quantity);  
  
        }  
  
        // Save the updated inventory.  
  
        Inventory.getInstance().saveInventoryToFile();  
  
    }  
  
}
```

```

        // Cancel the order and refresh the order history.

        order.cancelOrder();

        JOptionPane.showMessageDialog(frame, "Order canceled
successfully! Your money will be refunded.");

        new OrderHistoryGUI(customer);

        frame.dispose();

    } else {

        JOptionPane.showMessageDialog(frame, "Order cannot be canceled
as it is already processed.");

    }

}

```

- When the "Cancel" button is clicked, this method is triggered:
 - It checks if the order has been processed. If it has, the cancellation is not allowed.
 - If the order is not processed, it:
 - Updates the inventory by returning the medicine stock to the inventory.
 - Saves the updated inventory.
 - Cancels the order and refreshes the order history.
 - Displays a success message to the user.

Summary:

The `OrderHistoryGUI` class provides a user interface for customers to view and manage their order history. The main components of the GUI include:

- A frame to display the title and order history.

- A scrollable panel to show the list of orders.
- Options to track or cancel individual orders.

The class integrates action listeners for tracking and canceling orders. The cancel functionality updates the inventory and ensures that orders can only be canceled if they haven't been processed yet.